

Lecture 13: Data Exploration

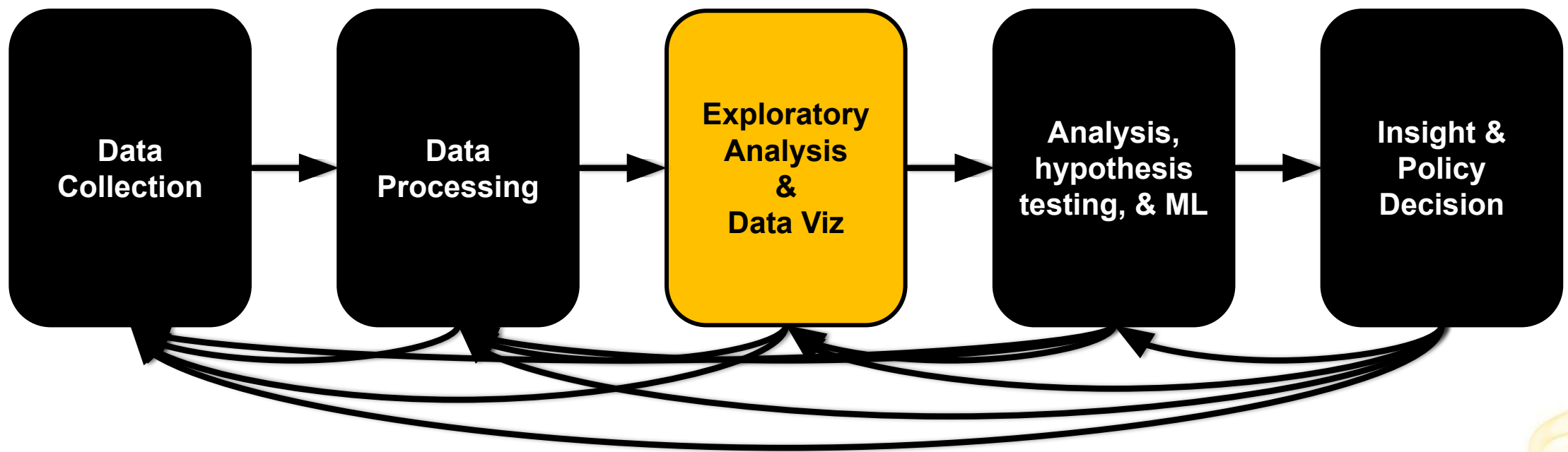
CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA3, WA3 were due yesterday
 - PA4, WA4 out tomorrow
 - Mid-semester check-in assignment available this week
 - Also asks about office hour preferences
- 

Data Exploration



Data Exploration - Terminology

This step is also commonly called **Exploratory Data Analysis**, or **EDA** for short.

You can use EDA and Data Exploration interchangeably for our purposes.



What is Data Exploration?

Data Exploration is the process of **getting to deeply know the data** you have. It often involves transforming, visualizing, and summarizing the data in order to:

1. Build and/or confirm **understanding** of the dataset and what/who created it
 - Summarize the main characteristics
 - Gain a visual understanding
2. **Inform** the subsequent analysis
 - Discover patterns and relationships
 - Spot anomalies
 - Check assumptions
3. **Iterate** on previous DS steps if needed
 - Identify any potential issues not already addressed
 - Modify or reinvent your hypothesis (⚠ be careful!)



Keep an open mind

Be willing to:

- Have your assumptions disproven
- Find something surprising
- Discover that you don't have the data you need
- Iterate
- ...more



*“Exploratory data analysis [Data Exploration] is an **attitude**, a **state of flexibility**, a **willingness to look** for those things that we believe are not there, as well as those that we believe to be there.”*

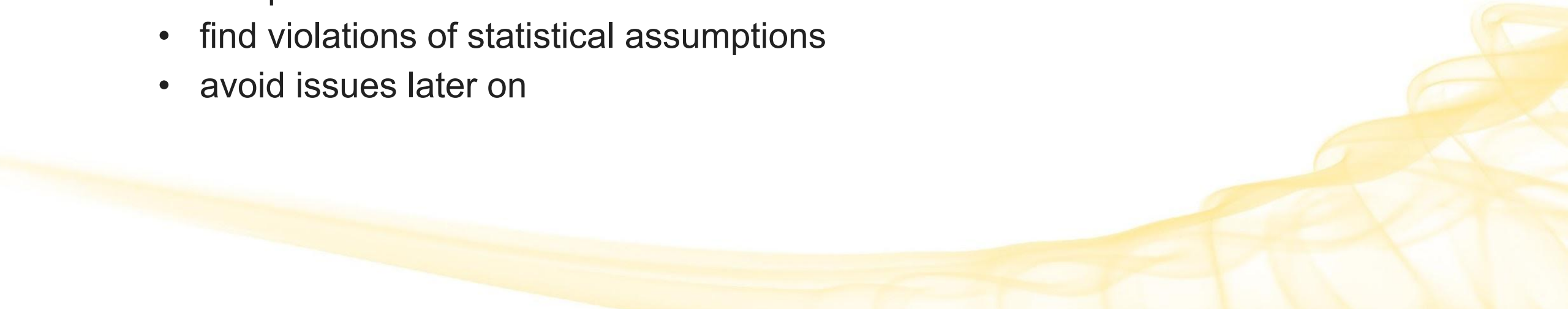
- John Tukey, American Mathematician and Statistician, and early Data Scientist



Why explore the data first?

It is a good practice to **understand the data** and **try to gather as many insights** from it before trying to test your hypothesis or run machine learning models against it.

This helps us:

- catch mistakes
 - see patterns in the data
 - find violations of statistical assumptions
 - avoid issues later on
- 

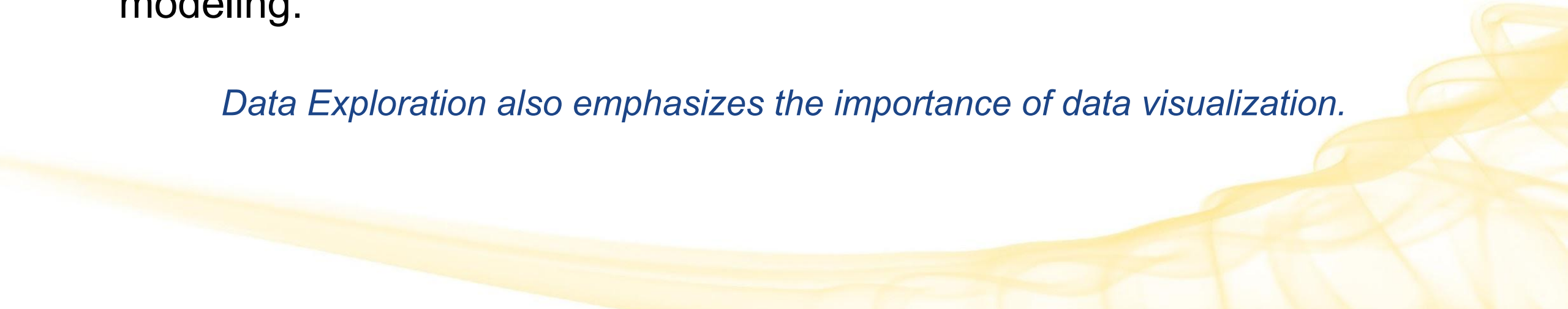
Data Exploration Process

Involves a series of steps and techniques to:

- Systematically explore the data
- Uncover hidden patterns
- Gain insights

This process ensures that you **don't miss critical information** and **can make informed decisions** about subsequent analyses or modeling.

Data Exploration also emphasizes the importance of data visualization.



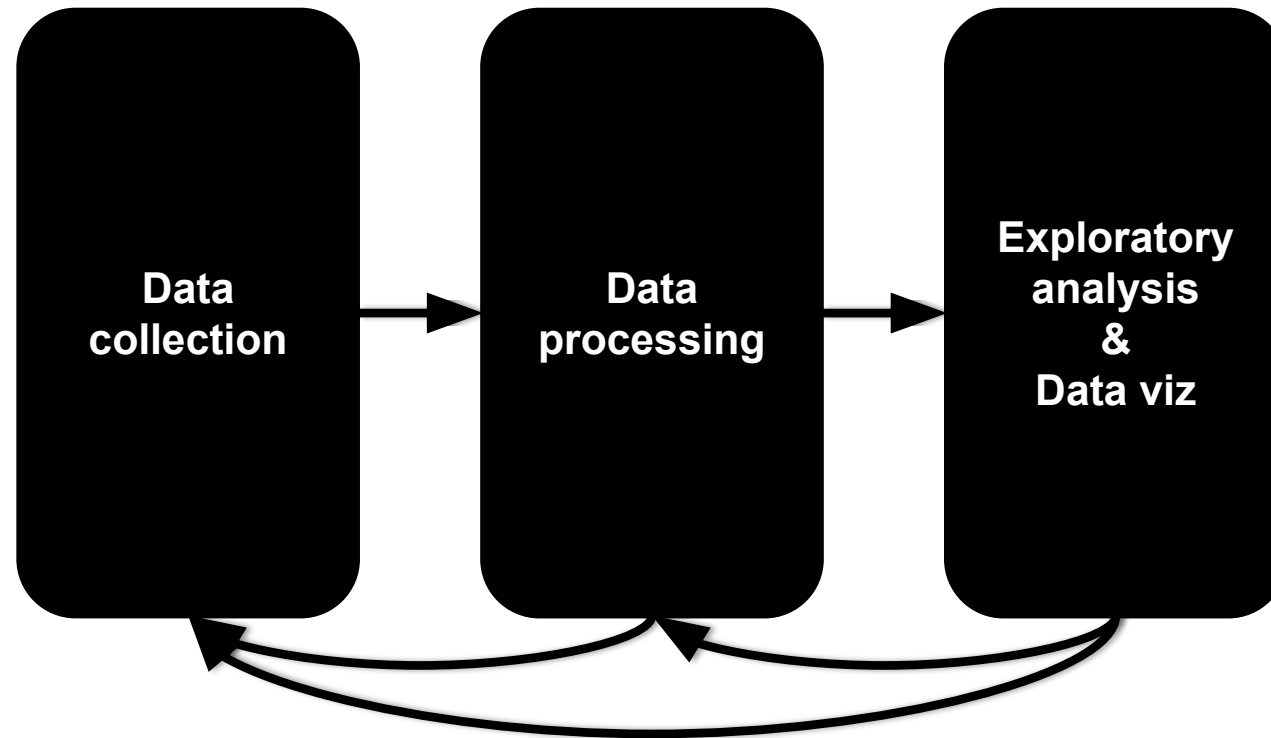
Data Exploration: Common Approach

Ask questions
about the data



Probe,
visualize, and
understand the
answer

Remember: Data Science is iterative!

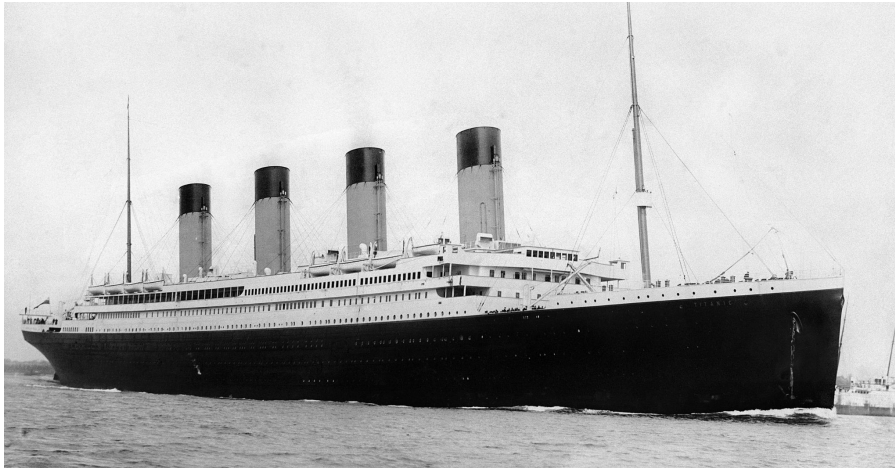


Motivating Example

Titanic

The Titanic was a British luxury passenger liner that sank in the North Atlantic Ocean on April 15, 1912, killing 1496 out of 2224 passengers and crew.

We have data from this famous event.




INTERNATIONAL MERCANTILE MARINE LINES
ALL OUR STEAMERS HAVE WIRELESS TELEGRAPH AND SUBMARINE SIGNALS

The Largest Steamers in the World

New
45,000
Tons
Each

OLYMPIC
TITANIC

Sell
from
NEW
YORK

April 13³ P. M.
MAY 4, 23,
JUNE 13, JULY 6,
APRIL 20¹² Noon
May 11, June 1
June 22, July 13

**FRENCH A LA CARTE RESTAURANT, TURKISH & ELECTRIC BATHS,
SWIMMING POOL, FOUR ELEVATORS, GYMNASIUM,
VERANDAH CAFE, PALM COURT, SQUASH RACQUET COURT.**

NEARBY SAILINGS
Baltic..... April 11, 12:00 Noon
Vaderland.... April 13, 10:00 A. M.
Minnewaska.. April 14.
Cedric..... April 18, 12:00 Noon

AMERICAN Pier 63, N. R.
9:30 A. M.
Plymouth—Cherbourg—Southampton
St. Paul..... May 4—St. Louis..... May 11

ATLANTIC TRANSPORT
New York—London Direct, Pier 58, N.R.
Minnewaska..... Apr. 14 | Minnehaha, May 11
Minnetonka, May 4, 10:26 AM | Minnewaska, May 18

RED STAR Pier 61, N. R.
10 A. M.
London, Paris via Dover—Antwerp
Vaderland..... Apr. 12 | Kronland..... Apr. 27
Lapland..... Apr. 29 | Finland..... May 4

PASSENGER DEPARTMENT, 9 BROADWAY, NEW YORK.

WHITE STAR - Piers 59 & 60, N. R.
Plymouth—Cherbourg—Southampton
(Olympic (new) Apr. 12, 3 PM | Olympic (new) May 4
Titanic (new) Apr. 20, 12 Noon | Titanic (new) May 11
New York—Queenstown—Liverpool:
Baltic..... Apr. 11, noon | Cedric..... Apr. 23
Cedric..... Apr. 18, noon | Adriatic..... May 3
FROM BOSTON To The MEDITERRANEAN
CANOPIC, Apr. 27, 3 PM. GRETO, May 18, 11 AM

CANADIAN SERVICE.

LARGEST STEAMERS FROM CANADA

MONTREAL VIA QUEBEC TO LIVERPOOL.

- *CANADA May 4, June 1, June 29
- *MEGANTIC May 11, June 8, July 6
- *TELTONIC May 18, June 15, July 13
- *LAURENTIC May 25, June 22, July 20

*Only One Class of Cabin (11) Passengers.

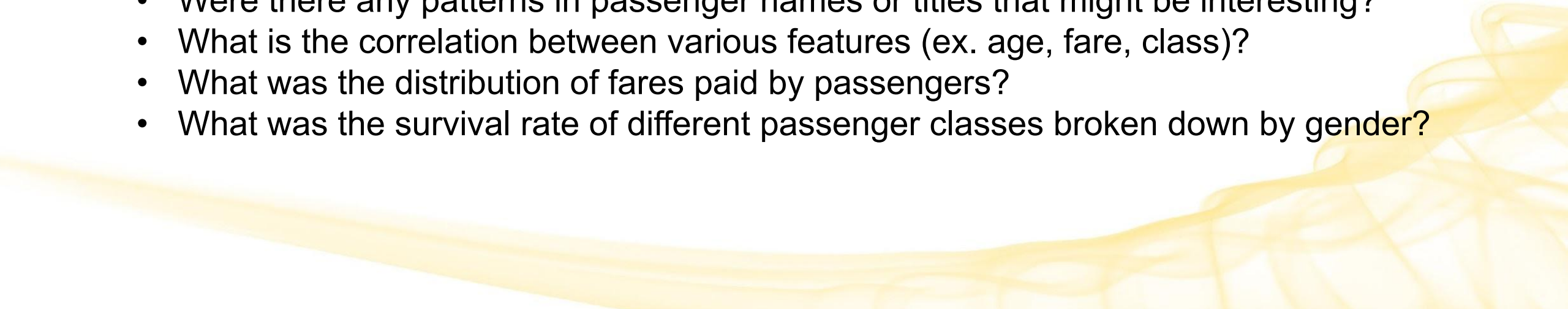


Titanic

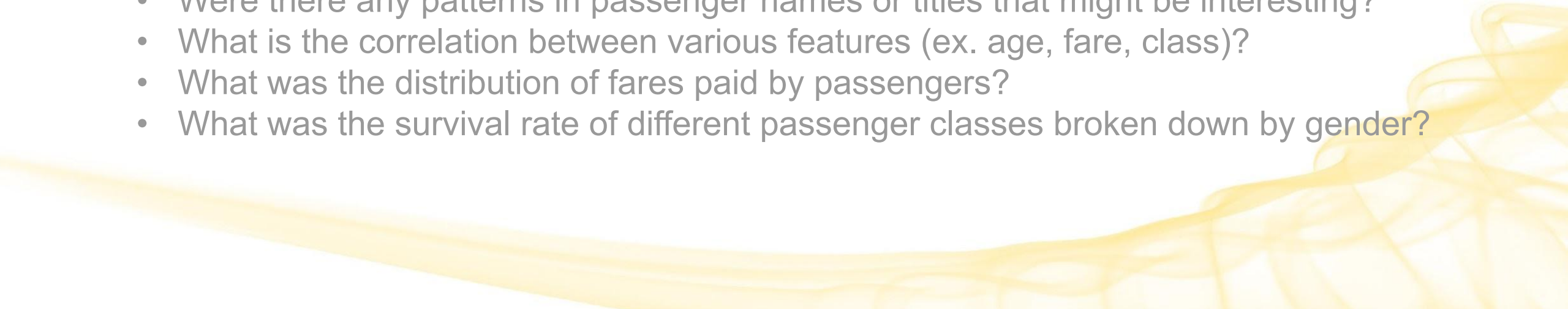
Nice dataset because it has real-world relevance and diverse features.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

We can ask explorational questions

- What is the demographic breakdown of passengers?
 - What was the overall survival rate of passengers?
 - Did age have an impact on survival rates?
 - Were passengers in higher classes more likely to survive?
 - What was the distribution of fares paid by passengers?
 - Were families more likely to survive together?
 - Did the point of embarkation impact survival rates?
 - How did gender influence survival rates?
 - Were there any patterns in passenger names or titles that might be interesting?
 - What is the correlation between various features (ex. age, fare, class)?
 - What was the distribution of fares paid by passengers?
 - What was the survival rate of different passenger classes broken down by gender?
- 

We can ask explorational questions

- What is the demographic breakdown of passengers?
 - What was the overall survival rate of passengers?
 - **Did age have an impact on survival rates?**
 - **Were passengers in higher classes more likely to survive?**
 - What was the distribution of fares paid by passengers?
 - Were families more likely to survive together?
 - Did the point of embarkation impact survival rates?
 - **How did gender influence survival rates?**
 - Were there any patterns in passenger names or titles that might be interesting?
 - What is the correlation between various features (ex. age, fare, class)?
 - What was the distribution of fares paid by passengers?
 - What was the survival rate of different passenger classes broken down by gender?
- 

We can also modify/refine these questions

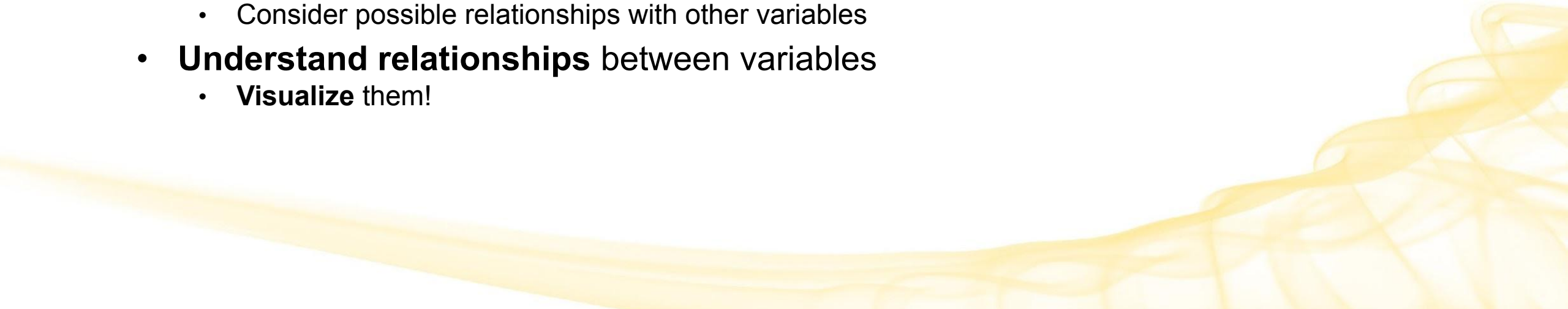
Initial Question	Modified Question
Did age have an impact on survival rates?	How does age impact the likelihood of survival?
Were passengers in higher classes more likely to survive?	Do first-class passengers have a higher survival rate compared to third-class passengers?
How did gender influence survival rates?	Is there a significant difference in survival rates between males and females?

Other kinds of questions we could ask

- Were there any **outliers** in the dataset, especially regarding fares or ages?
 - Did passengers with **missing values** for certain features have different survival rates?
- 

Data Exploration Process

Data Exploration Steps

- Gather a list of **initial questions**
 - Ground questions in your initial hypotheses and domain knowledge
 - Refine your questions to gain a deeper understanding of the factors, if needed
 - Check for any **missing data** or **extreme outliers**
 - Anything that may have been missed in Data Pre-Processing
 - Explore **characteristics of the data**
 - What does the data represent? What does each row represent?
 - What data types are present?
 - What variables do you have and what do they represent?
 - **Analyze each variable** in the dataset
 - Understand them fully - what they are, how they are measured, their distribution, etc
 - Consider possible relationships with other variables
 - **Understand relationships** between variables
 - **Visualize** them!
- 

Do you have the right data?

Does the data you have appear to contain the information needed to address your hypothesis?

- Is it **expansive enough**?
 - ex. Looking at grades across all Data Science classes but data is only from 602
- Does it cover the **timeframe** you need (if time-specific)?
 - ex. Need 2006 but data is from 1992
 - ex. Looking at seasonal trends at end of year but only have summer data
 - Pro-tip: watch out for time zones when working with timestamps!
- Does it cover the **location** you need (if location-specific)?
 - ex. Need Massachusetts data but only have Boston suburbs
- Was it sampled from the **population** you are interested in?
 - ex. Need data on Women over 50 but only have Men over 50

General considerations

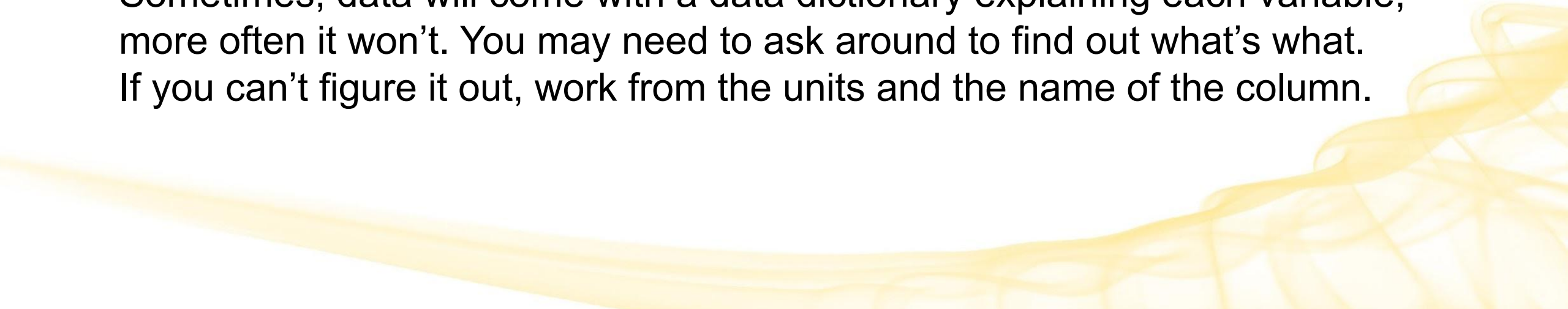
- **Structure:** the shape of the dataset
 - **Granularity:** how fine/coarse is each variable and the overall data
 - **Scope:** how complete or incomplete is the data
 - **Temporality:** how is the data situated in time
 - **Faithfulness:** how well does the data capture reality
- 

Getting an overview of the data

Once your dataframe is loaded, you'll be able to look at each column and its datatype

- `df.shape` tells you the overall structure/shape of your data
- `df.columns` lists all the columns
- `df.dtypes` tells you the datatypes of each column

Sometimes, data will come with a data dictionary explaining each variable; more often it won't. You may need to ask around to find out what's what. If you can't figure it out, work from the units and the name of the column.



Understanding Records

What does each record in the dataset represent?

- Examples
 - Sensor data captured at a particular time?
 - A single person and information about them?
 - A sale of a product?

Sometimes peaking at the first or last few records is enough to get a sense:

```
df.head(3)
```

	Unnamed: 0	date	births	wday	year	month	day_of_year	day_of_month	day_of_week
0	1	2015-01-01	8068	Thu	2015	1	1	1	5
1	2	2015-01-02	10850	Fri	2015	1	2	2	6
2	3	2015-01-03	8328	Sat	2015	1	3	3	7

Basic Data Manipulation

Some data might be in formats that are hard to work with.

Feel free to add your own variables (ex. columns):

- **date** might become **date_in_days_since_zero**
- Survey results, such as **strong agree, agree, neutral, disagree, strong disagree**, might become **2, 1, 0, -1, -2**

Similarly, feel free to change units:

- **Days** can become **seconds**
 - **Pounds** can become **grams**
- 

Learning About Individual Variables



Variable Units

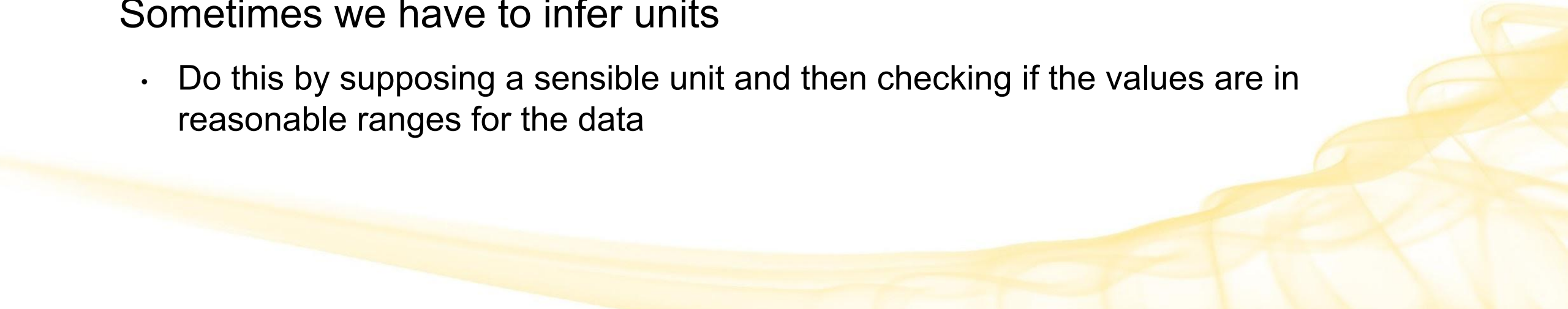
Are the units of the variable clear?

- For categorical data, is it clear what the categories represent?
- For measurements like weight, are they in english units or metric units (lbs vs kgs)?

Often units are made clear by:

- The header row name (ex. `height_in_inches`, `milliseconds_since_epoch`)
- A README file included with the dataset
- Asking someone familiar with the data

Sometimes we have to infer units

- Do this by supposing a sensible unit and then checking if the values are in reasonable ranges for the data
- 

Understanding Variables

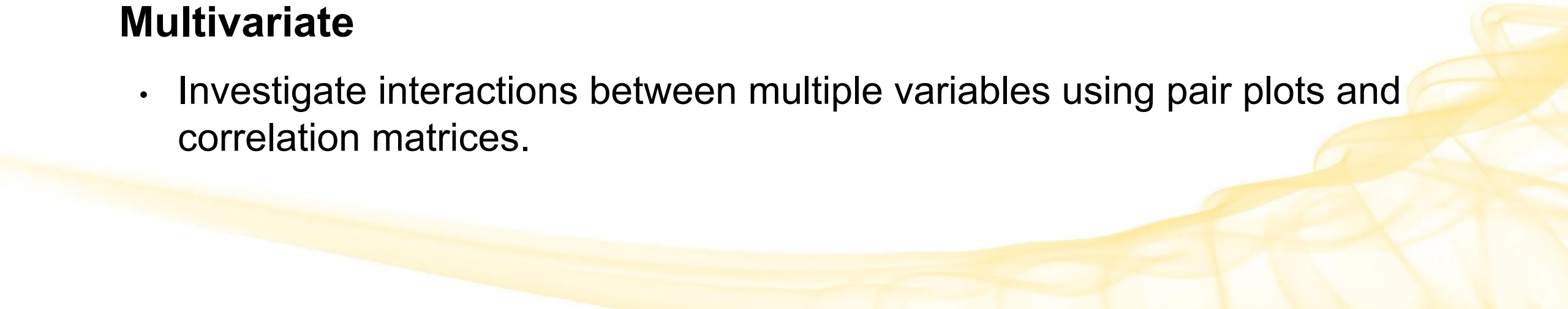
Univariate

- Analyze individual variables using summary statistics and visualizations such as histograms and box & whisker plots.

Bivariate

- Analyze the relationship between two variables with scatter plots, pairwise correlation coefficients, and cross-tabulations.

Multivariate

- Investigate interactions between multiple variables using pair plots and correlation matrices.
- 

Variable Summary Statistics

- We can use the **describe** method to generate summary statistics for individual columns in the dataset as part of our exploration.
- Ex. `df["VARIABLE_NAME"].describe()`



```
df["Species"].describe()
```



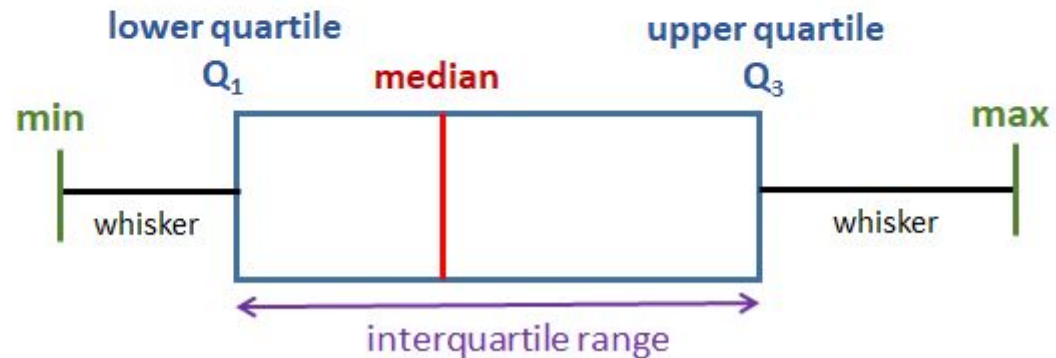
Species

count	150
unique	3
top	Iris-setosa
freq	50

dtype: object

Variable Visualization: Box & Whisker Plots

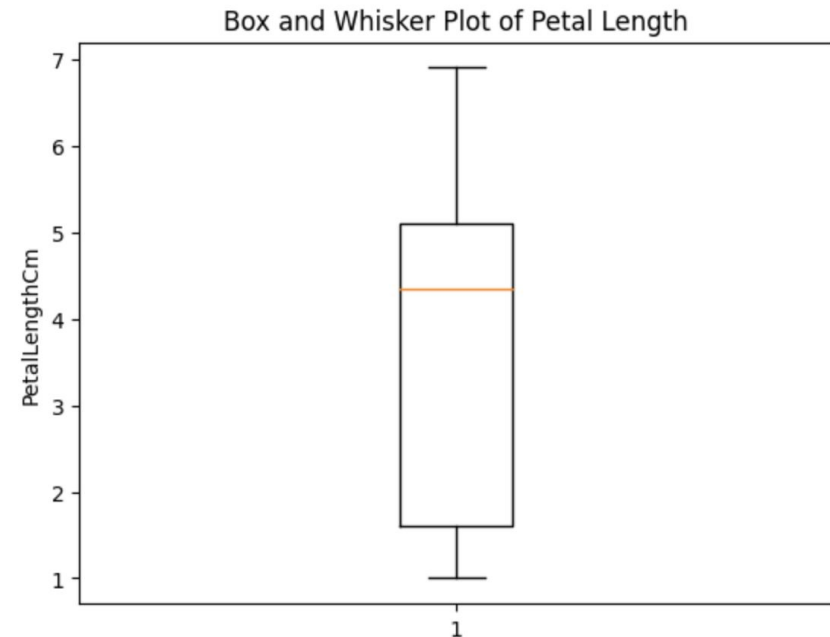
We can also create **box and whisker plots** to visualize the distribution of numeric variables.



```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.boxplot(df['PetalLengthCm'].dropna())
ax.set_title('Box and Whisker Plot of Petal Length')
ax.set_ylabel('PetalLengthCm')

plt.show()
```



Variable Distribution of Values

You might want to know about the frequencies/rough distribution of the variables.

For **categorical** variables, `df["VARIABLE_NAME"].value_counts()` is often enough

```
df['Marital Status'].value_counts()
```



count	
Marital Status	
Married	535
Single	458

```
df['Occupation'].value_counts()
```



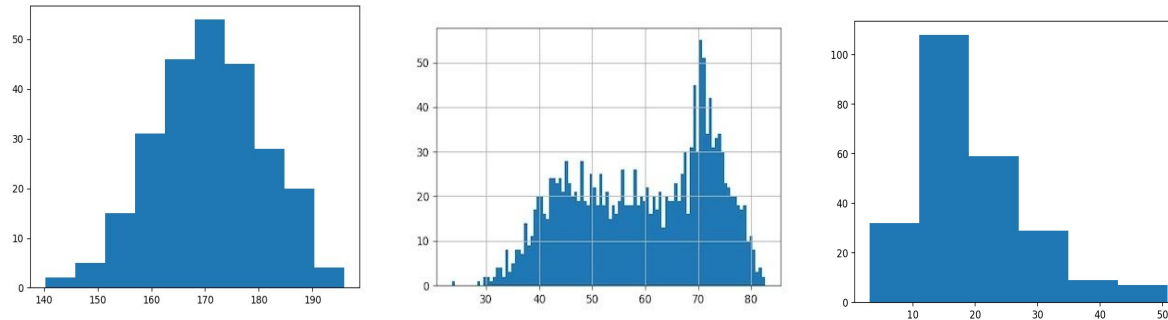
count	
Occupation	
Professional	276
Skilled Manual	255
Clerical	177
Management	173
Manual	119

Variable Distribution of Values

If you have a more **continuous** variable, you can visualize the distribution using tools like histograms.

Ex. `df["VARIABLE_NAME"].hist()` will usually work:

```
df['PetalLengthCm'].hist()
```



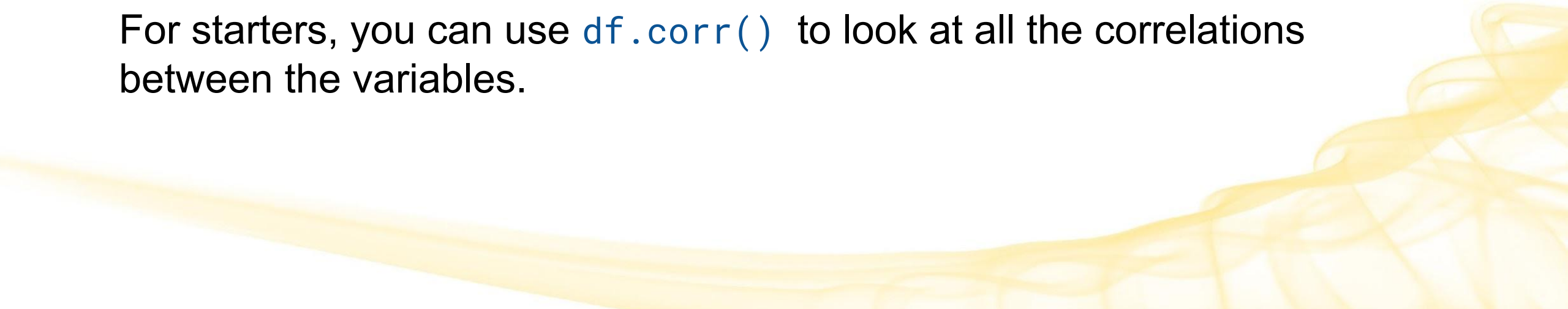
Different variable histograms tell different stories!

Variable Relationships

It is helpful to **visualize relationships** between variables.

- Use graphs, charts, and plots to explore and understand how two or more variables are related or correlated.
- Helps us see patterns, trends, and associations between variables in a graphical and intuitive way.

For starters, you can use `df.corr()` to look at all the correlations between the variables.

A decorative graphic consisting of several overlapping, wavy, yellow lines that sweep across the bottom right corner of the slide, creating a sense of movement and flow.

Variable Correlation in Pandas

The `df.corr()` method in pandas calculates the **pairwise correlation coefficients** between all numerical columns in a dataframe.

By default, it calculates Pearson correlation coefficients, which measure the linear relationship between two variables:

$$r = \frac{\sum (x_i - \bar{x}) (y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Pearson correlation coefficient ranges from -1 to 1, where:

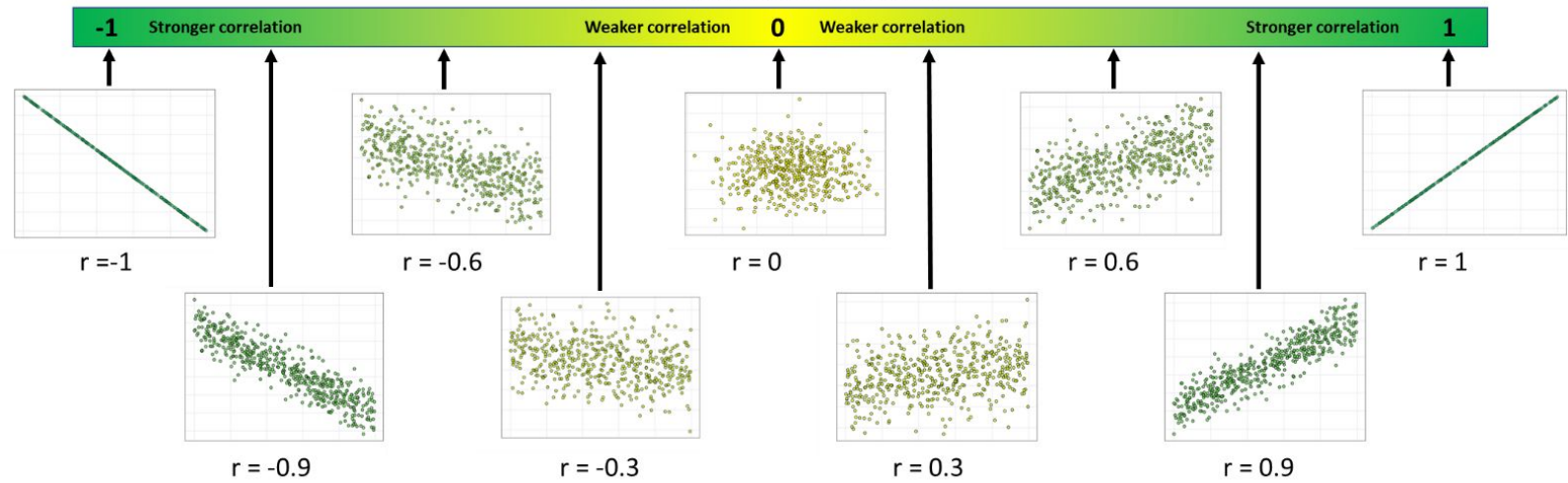
- 1 indicates a perfect positive linear relationship (**perfectly correlated**)
- -1 indicates a perfect negative linear relationship (**perfectly anti-correlated**)
- 0 indicates **no linear relationship**.

The Pearson correlation coefficient (r)

The Pearson correlation coefficient (r) is the most common way of measuring a linear correlation.

It is a number between -1 and 1 that measures the strength and direction of the relationship between two variables.

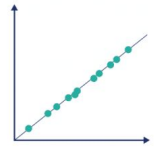
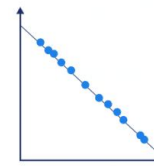
$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$



Using r

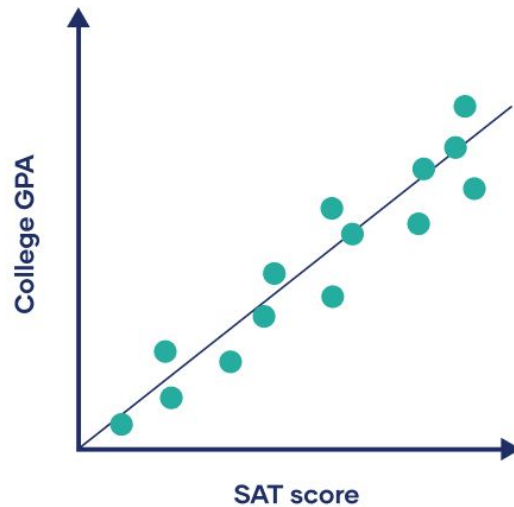
Remember, the correlation coefficient represents a number from -1 to 1 that tells you the **strength** and **direction** of a relationship between variables.

- it reflects how similar the measurements of two or more variables are across a dataset.

r	Correlation Type	Meaning	Visual
1	Perfect positive correlation	When one variable changes, the other variable(s) change in the same direction	
0	Zero correlation	There is no relationship between the variables	
-1	Perfect negative correlation	When one variable changes, the other variable(s) change in the opposite direction	

Correlation Example

You gather a sample of 5,000 college graduates and survey them on their high school SAT scores and college GPAs. You visualize the data to check for a linear pattern:

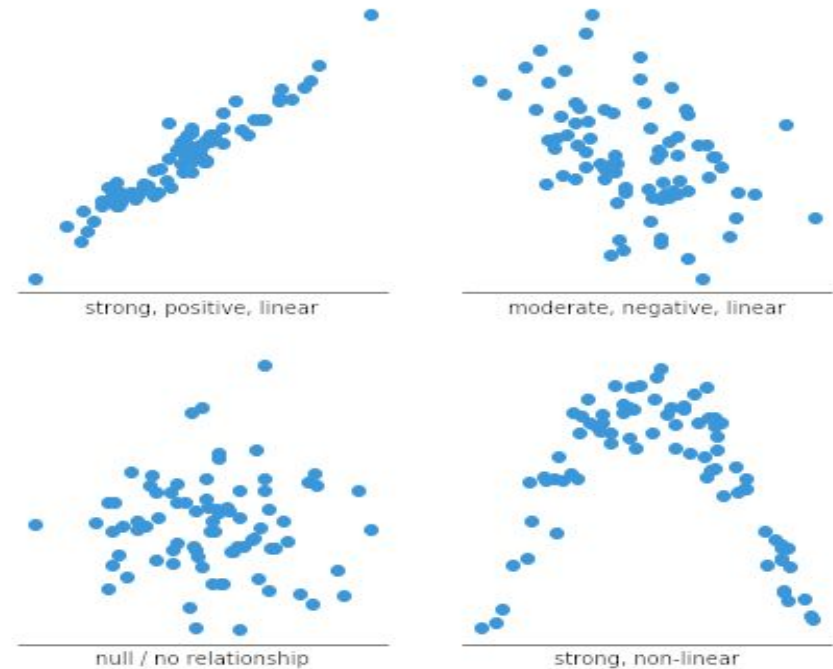


Variable Relationships: Scatter Plot

Again, scatter plot is a very simple technique that can tell you a lot about a variable or set of variables.

It's particularly helpful for quickly checking relationships between variables.

Ex. `df.plot.scatter("col1", "col2")`



Relationships between categorical variables

Another way to view relationships between variables is to use **cross-tabulation** to compare categorical data with `pd.crosstab()`

Ex.

```
pd.crosstab(df['state_name'], df['magnitude'])
```

magnitude	0	1	2	3	4	5
state_name						
Alabama	108	137	71	20	10	2
Arizona	3	0	1	0	0	0
Arkansas	38	123	39	15	4	0
California	16	7	3	0	0	0
Colorado	9	18	6	4	0	0
Connecticut	4	6	0	0	2	0
Florida	148	82	23	8	0	0
Georgia	114	156	59	25	2	0



Pair plots

Pair plots are useful particularly when looking for relationships between several variables.

There is a nice python visualization library,  **seaborn**, that makes this easy.

Just use `sns.pairplot()` to plot pairwise relationships.

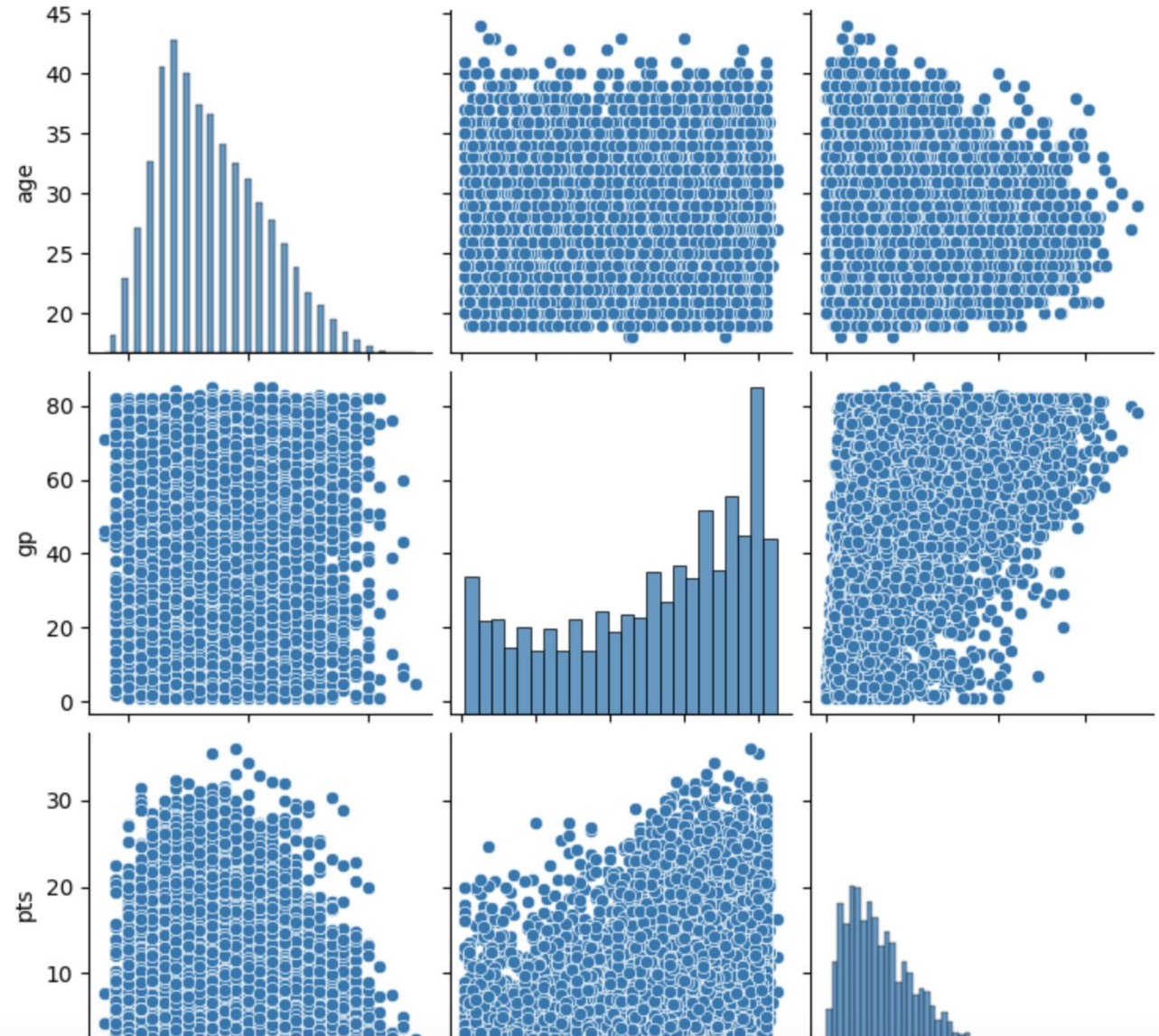
Row → x axis
Column → y axis

```
import pandas as pd
import seaborn as sns

df = pd.read_csv('nba_all_seasons.csv')

sns.pairplot(df, vars = ["age", "gp", "pts"], dropna = True)
```

<seaborn.axisgrid.PairGrid at 0x78a9ee3350c0>



Correlation Matrices

`df.corr()` can be done for higher numbers of variables as well.

```
df.corr(numeric_only=True)
```

Ex.

	Unnamed: 0	age	player_height	player_weight	gp	pts	reb	ast	net_rating	oreb_pct
Unnamed: 0	1.000000	-0.136497	-0.062835	-0.081557	-0.085773	0.047024	-0.002010	0.030067	-0.000485	-0.164933
age	-0.136497	1.000000	-0.007904	0.063561	0.057442	0.011353	0.037386	0.092359	0.092896	-0.055228
player_height	-0.062835	-0.007904	1.000000	0.822141	0.004963	-0.055284	0.424220	-0.442781	-0.003074	0.589538
player_weight	-0.081557	0.063561	0.822141	1.000000	0.022828	-0.025023	0.438112	-0.371675	0.003547	0.599511
gp	-0.085773	0.057442	0.004963	0.022828	1.000000	0.536003	0.471405	0.382726	0.248412	-0.014763
pts	0.047024	0.011353	-0.055284	-0.025023	0.536003	1.000000	0.624509	0.664320	0.215199	-0.125164
reb	-0.002010	0.037386	0.424220	0.438112	0.471405	0.624509	1.000000	0.247841	0.187820	0.407960
ast	0.030067	0.092359	-0.442781	-0.371675	0.382726	0.664320	0.247841	1.000000	0.162865	-0.337970
net_rating	-0.000485	0.092896	-0.003074	0.003547	0.248412	0.215199	0.187820	0.162865		
oreb_pct	-0.164933	-0.055228	0.589538	0.599511	-0.014763	-0.125164	0.407960	-0.337970		

A quick note on patterns



Commonly used syntax patterns for df

You can **apply functions directly** to a dataframe column:

```
df[column_name].function()
```

Ex.

```
df['births'].sum()
```

```
3978497
```



Commonly used syntax patterns for df

You can also **apply custom functions** to a dataframe variable:

```
df['column_name'].apply(custom_function)
```

Ex.

```
# Custom function definition
def double_value(value):
    return value * 2

# Apply the custom function to the 'births' column and
# assign the results to a new column 'doubled_values'
df['doubled_values'] = df['births'].apply(double_value)

# You can also use an anonymous function (lambda)
# for one-time needs
df['doubled_values'] = df['births'].apply(lambda x: x*2)
```

Commonly used syntax patterns for df

You can also easily **filter data**, and there are many nice ways to do so.

One way is to filter df rows by a condition:

- filter the dataframe based on the condition
- apply statistical functions to that specific column

Pattern

```
df[df['column_name'] > value]
```

a boolean condition

```
df[df['condition']][column_name].statistics_function()
```

Example

```
# Show how many births there were in October  
df[(df['day_of_year'] > 274) & (df['day_of_year'] < 304)]['births'].sum()
```

```
317915
```

Commonly used syntax patterns for df

You can **aggregate** data and **calculate statistics** within each group:

```
df.groupby('condition')[column_name].statistics_function()
```

Ex.

```
df.groupby('state_name')['injured_count'].mean()
```



injured_count

state_name

Alabama	9.681034
Arizona	0.000000
Arkansas	3.757991
California	0.192308
Colorado	2.675676
Connecticut	45.250000

Commonly used syntax patterns for df

You can also perform **in-place operations** on a variable.

In-place operation

```
df['column_name'].function(value, inplace=True)
```

Non-in-place operation (default behavior)

```
df_copy = df['column_name'].function(value, inplace=False)
```

